

Họ và tên: Đặng Xuân Bách

MSSV: 25520109

BÀI LÀM: CÁC CĂN CỨ VÀ Ý TƯỞNG CẢI TIẾN CỦA CÁC THUẬT TOÁN SẮP XẾP

1. Nhóm cải tiến dựa trên kỹ thuật Đổi chỗ (Interchange/Exchange)

- **Bubble Sort -> Shaker Sort:**

- **Căn cứ:** Thuật toán Bubble Sort (Sắp xếp nổi bọt) có nhược điểm. Các phần tử lớn được đưa về cuối mảng rất nhanh, nhưng các phần tử nhỏ ở sai vị trí (nằm cuối mảng) lại di chuyển về đầu rất chậm (mỗi lần lặp chỉ nhích lên 1 vị trí).
- **Ý tưởng cải tiến:** Shaker Sort khắc phục bằng cách duyệt mảng theo **hai chiều**. Sau khi đẩy phần tử lớn nhất về cuối, thuật toán quay ngược lại để đưa phần tử nhỏ nhất về đầu ngay trong lượt đó. Việc này giúp các phần tử nhỏ về đúng vị trí nhanh hơn đáng kể.

- **Interchange Sort -> QuickSort:**

- **Căn cứ:** Interchange Sort so sánh và đổi chỗ các cặp phần tử một cách cục bộ với độ phức tạp $O(N^2)$, không hiệu quả với dữ liệu lớn.
- **Ý tưởng cải tiến:** QuickSort thay đổi cách tiếp cận bằng tư duy **Chia để trị (Divide and Conquer)**. Thay vì đổi chỗ ngẫu nhiên, thuật toán chọn một phần tử làm "chốt" (pivot) để phân hoạch mảng thành hai phần riêng biệt: một phần nhỏ hơn chốt và một phần lớn hơn chốt. Việc này giúp giảm không gian tìm kiếm và đưa độ phức tạp trung bình xuống $O(N \log N)$

2. Nhóm cải tiến dựa trên kỹ thuật Chèn (Insertion)

- **Insertion Sort -> Binary Insertion Sort:**

- **Căn cứ:** Trong Insertion Sort cổ điển, để tìm vị trí chèn cho phần tử hiện tại vào dãy con đã sắp xếp phía trước, thuật toán phải duyệt tuần tự từ trên xuống, mất $O(N)$ cho mỗi lần tìm.
- **Ý tưởng cải tiến:** Tận dụng tính chất dãy con phía trước đã được sắp xếp, Binary Insertion Sort sử dụng thuật toán **Tìm kiếm nhị phân (Binary Search)** để xác định vị trí chèn. Điều này giảm số lần so sánh từ $O(N)$ xuống $O(\log N)$, dù số lần di chuyển phần tử vẫn không đổi.

- **Insertion Sort -> Shell Sort:**

- **Căn cứ:** Insertion Sort chỉ phát huy tối đa hiệu quả khi mảng "gần như đã được sắp xếp". Nếu phần tử nhỏ nằm sai vị trí quá xa, chi phí di chuyển là rất lớn.
- **Ý tưởng cải tiến:** Shell Sort thực hiện "sắp xếp sơ bộ" bằng cách so sánh các phần tử cách nhau một khoảng h thay vì các phần tử liền kề. Khoảng h giảm dần về 1. Khi h mảng đã trở nên "gần như sắp xếp", lúc này thuật toán trở về là Insertion Sort nhưng chạy với tốc độ tối ưu nhất.

3. Nhóm cải tiến dựa trên kỹ thuật Chọn (Selection)

- **Selection Sort -> Heap Sort:**

- **Căn cứ:** Selection Sort tốn chi phí $O(N)$ để duyệt toàn bộ mảng con nhằm tìm ra phần tử lớn nhất/nhỏ nhất ở mỗi bước lặp. Đây là thao tác lặp lại và tốn kém nhất.
- **Ý tưởng cải tiến:** Heap Sort cải tiến việc tìm kiếm này bằng cách tổ chức dữ liệu dưới dạng cấu trúc cây **Heap (Vun đống)**. Trên Heap, phần tử lớn nhất luôn nằm ở gốc, cho phép truy xuất trong $O(1)$ và tái cấu trúc cây chỉ mất $O(\log N)$. Tổng thể thuật toán đạt hiệu suất $O(N \log N)$ ổn định.

4. Nhóm cải tiến dựa trên kỹ thuật Trộn (Merge)

- **Merge Sort -> Natural Merge Sort:**

- **Căn cứ:** Merge Sort cơ bản thực hiện chia đôi mảng một cách máy móc bất kể dữ liệu bên trong có thể đã có thứ tự sẵn.
- **Ý tưởng cải tiến:** Natural Merge Sort tận dụng các "**đường chạy tự nhiên**" (**natural runs**) – là các đoạn con liên tiếp đã tình cờ được sắp xếp tăng dần trong dữ liệu gốc. Thuật toán chỉ thực hiện trộn các đoạn này lại với nhau, giúp giảm thiểu số lần tách/trộn không cần thiết nếu dữ liệu đã có độ sắp xếp nhất định.

KẾT LUẬN

Sự phát triển của các thuật toán sắp xếp dựa trên nguyên tắc tối ưu hóa tài nguyên tính toán: chuyển từ độ phức tạp $O(N^2)$ -> $O(N \log N)$ bằng các kỹ thuật như **Chia để trị** (QuickSort, MergeSort), sử dụng **Cấu trúc dữ liệu thông minh** (HeapSort), hoặc **Bước nhảy khoảng cách** (ShellSort).